

job_ops-mcp

End-to-End User Guide

A self-hosted MCP server for the full job-search loop — portal scanning, JD evaluation, tailored resume + cover PDFs, outreach drafting, story bank, negotiation brief.

The chat reasons. The server executes. Every artifact comes back as a localhost link.

Version	0.3.0
Repo	https://github.com/mohith-das/job_ops-mcp
License	MIT

Generic content. Examples use a fictional candidate.
Not affiliated with or endorsed by santifer/career-ops.

Contents

1	What this is, and the mental model	2
2	Architecture	2
3	Installation and setup	3
4	The data model	7
4.1	Relational shape	8
4.2	The job lifecycle (state machine)	9
5	Tool reference	9
6	Worked end-to-end examples	13
6.1	Example A: paste a JD, get a tailored PDF, mark applied	13
6.2	Example B: scan portals, triage, batch evaluate	15
6.3	Example C: find a warm intro, draft outreach, follow up	16
7	The daily operating ritual	18
8	Customization	19
9	The hard rules (safety model)	21
10	Troubleshooting and FAQ	22
11	This finds jobs. It doesn't send them.	23

1 What this is, and the mental model

job_ops-mcp is a self-hosted **Model Context Protocol** server that exposes the full job-search workflow to your chat client as 37 tools and 6 editable behaviour-shaping resources. The chat client (Claude Desktop, LibreChat, Cursor, any MCP-aware client that speaks streamable-HTTP) does the reasoning; the server executes the mechanical work and returns every generated artifact as an `http://localhost:7891/...` link the chat can paste straight into the conversation.

The split is the whole point.

The chat thinks.

Reads your rubric and your career packet. Scores a JD. Drafts the 6-block evaluation report. Picks bullets per role type. Writes the warm-intro DM. Decides what to ship.

The server executes.

Persists jobs to a single SQLite file with a known schema. Polls Greenhouse / Ashby / Lever / Workday APIs. Renders Chromium HTML→PDF. Validates outreach drafts against safety rails before saving. Serves every artifact at a stable local URL.

You decide and send.

Nothing auto-submits an application or auto-sends a DM. job_ops-mcp is preview-only at every irreversible boundary.

Where it comes from

job_ops-mcp merges two systems into one process. The 6-block evaluation report, the ATS-friendly HTML CV template, the unicode-normalization pipeline, and the negotiation playbook framing are ported from [santifer/career-ops](#) (MIT licensed). The three-dimension rubric (resume / taste / visa), the schema shape, the strict-JSON rater contract, the warm-intro and founder DM tone rules, and the H1B signal logic are distilled from a personal Postgres + n8n pipeline (“JSA”).

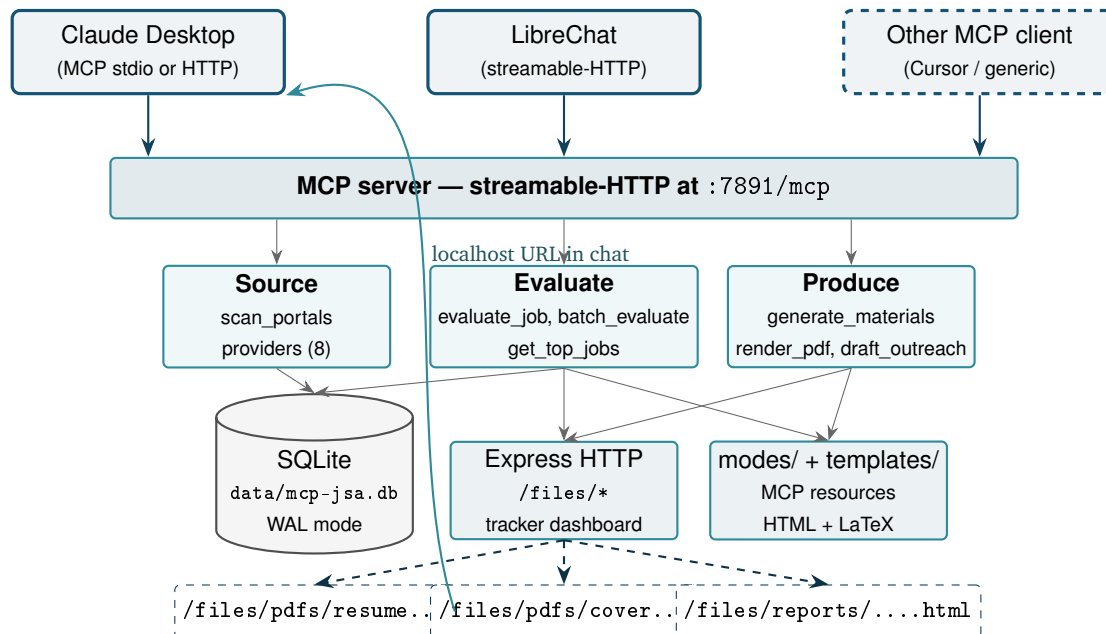
Not affiliated with or endorsed by santifer’s career-ops. This is an independent project that adapts the publicly-released MIT-licensed templates and rubric shape into the MCP transport surface. Credit where due, but no endorsement implied.

One sentence

Source jobs cheaply, evaluate them carefully, produce tailored materials, draft (don’t send) the outreach, track everything in one local database, do it every day.

2 Architecture

One process. Three planes. Six MCP resources and 37 tools.



Read it like this: the chat client posts a JSON-RPC `tools/call` to `/mcp`. The server routes to one of the three planes — **Source** (pull jobs from ATS APIs), **Evaluate** (score, write the 6-block report, persist to SQLite), or **Produce** (tailor bullets, render PDFs, draft outreach). The Express HTTP server on the same port serves every generated file at `/files/...` and a read-only tracker dashboard at `/`. The chat pastes those links back into the conversation — the human clicks to inspect.

Why one process

No external Postgres. No n8n. No cloud anything. The single-process design means:

- **Boot in 2 seconds.** `npx job_ops-mcp start` — no docker compose, no migrations to run by hand, no init container.
- **Local data, local artifacts.** Your jobs, your scores, your LinkedIn export, your H1B import, your tailored PDFs all live next to your `cv.md` in one directory. Back it up by copying that directory.
- **Chat-driven, not autonomous.** The server reacts to chat tool calls. It is not a background daemon scraping the web.
- **Opt-in cron.** If you do want background scans, `scheduler_enable` turns on a per-job interval. Off by default.

3 Installation and setup

Prerequisites

- **Node.js ≥ 20 .** Check with `node -version`. Install from <https://nodejs.org/> if missing.
- **An MCP-aware chat client.** Claude Desktop, LibreChat, Cursor, or any other client that supports streamable-HTTP MCP.

- **(Optional)** an LLM API key — Gemini (free tier covers ordinary use) or DeepSeek. Required only for the api/batch paths; the default chat mode of every tool works without one.

Five commands

```
# 1. Scaffold cv.md, config/profile.yml, portals.yml in the current directory
#    and create the SQLite database. Idempotent --- safe to re-run.
npx job_ops-mcp init

# 2. Edit the three files. Replace every <TODO> placeholder.

# 3. Rebuild the career_packet from your now-real cv.md. (Or just re-run
#    'init' --- it auto-reseeds when it detects cv.md changed.)
npx job_ops-mcp reseed

# 4. Confirm everything is wired.
npx job_ops-mcp doctor

# 5. Boot the server. Chromium auto-installs on first run (~150 MB, one-time).
npx job_ops-mcp start
```

TIP

The CLI is also published as the dual alias `npx job-ops-mcp` (hyphen-only). They are the same binary — use whichever your fingers prefer.

TIP: The edit-cv.md loop

Your `career_packet` is the source of truth the chat uses to score JDs and draft materials. It's built from `cv.md` + `config/profile.yml`. Whenever you edit `cv.md`, **re-run reseed** so the chat sees the new bullets. `reseed` writes a NEW active version (previous version demoted, history kept). The chat can also call the `reseed_career_packet` MCP tool to do the same thing without leaving the conversation. `doctor` warns when `cv.md` changed since the last reseed.

What init creates

After `npx job_ops-mcp init` from a clean directory you'll have:

```
my-job-search/
|-- cv.md                <- copy of cv.example.md
|-- config/
|   '-- profile.yml      <- copy of config/profile.example.yml
|-- portals.yml          <- copy of portals.example.yml
'-- data/
    '-- mcp-jsa.db       <- SQLite with both migrations applied
```

The three editable files are the “user layer.” The server reads them at runtime; you own their content.

`cv.md`

Plain Markdown. Standard headings (## Work Experience, ## Projects, ## Education, ## Skills). The renderer parses your bullets into structured experience entries. Keep bullets concrete (action verbs, real metrics).

`config/profile.yml`

Your identity, target roles + archetypes, narrative likes / dislikes, comp range, location, and visa status. The rubric reads this on every score, the materials generator reads it to populate the cover-letter header, and the warm-intro drafter pulls credibility markers from `narrative.superpowers`.

`portals.yml`

The companies whose ATS endpoints `scan_portals` should poll, plus title and location filters. Greenhouse / Ashby / Lever are auto-detected from any `careers_url`; Workday needs an explicit `workday_url`; Amazon / Google / generic Playwright are opt-in via `provider:`.

Environment variables

All optional. Defaults in parens.

Variable	Purpose
<code>MCP_JSA_PORT</code>	HTTP port. (7891)
<code>MCP_JSA_HOST</code>	Bind host. (127.0.0.1)
<code>MCP_JSA_PROJECT_ROOT</code>	Where <code>cv.md</code> / <code>profile.yml</code> / <code>portals.yml</code> live. (current directory)
<code>MCP_JSA_DATA_DIR</code>	SQLite location. (<project_root>/data)
<code>MCP_JSA_OUTPUT_DIR</code>	Where generated PDFs and report HTML land. (<project_root>/output)
<code>MCP_JSA_VISA_SCORING</code>	Set to false to drop the visa surface entirely (see §8). (true)
<code>MCP_JSA_LLM_PROVIDER</code>	gemini deepseek none. (gemini)
<code>MCP_JSA_LLM_MODEL</code>	Provider-specific model id. (empty → provider default)
<code>GEMINI_API_KEY</code>	Gemini key. Needed only for api/batch paths.
<code>DEEPSEEK_API_KEY</code>	DeepSeek key. Same.
<code>MCP_JSA_SCHEDULER_ENABLED</code>	Whether opt-in cron runs at all. (false)

Connecting your chat client

`npx job_ops-mcp connect` prints copy-paste-ready config for the common clients. Below are the two you most likely care about.

Claude Desktop (stdio transport)

Claude Desktop's local MCP only speaks **stdio**, not HTTP — the `-stdio` flag binds MCP to `stdin/stdout`. The HTTP file server still binds to `MCP_JSA_PORT` in the background so the `/files/*`

artifact links the chat returns still resolve in your browser.

Add to `~/Library/Application Support/Claude/claude_desktop_config.json` on macOS (or `%APPDATA%/Claude/claude_desktop_config.json` on Windows):

```
{
  "mcpServers": {
    "job_ops-mcp": {
      "command": "npx",
      "args": ["-y", "job_ops-mcp", "start", "--stdio"],
      "env": {
        "MCP_JSA_PORT": "7891",
        "MCP_JSA_PROJECT_ROOT": "/absolute/path/to/your/job-search/dir"
      }
    }
  }
}
```

Restart Claude Desktop. The 33–37 tools should appear in the tool picker.

LibreChat (host process)

In `librechat.yaml`, under top-level `mcpServers::`

```
mcpServers:
  job_ops-mcp:
    type: streamable-http
    url: http://127.0.0.1:7891/mcp
    timeout: 60000
```

Boot the server separately (`npx job_ops-mcp start`), then restart LibreChat.

LibreChat (Docker)

Inside a container, `127.0.0.1` points at the container itself — not your host. Two changes are required:

```
mcpServers:
  job_ops-mcp:
    type: streamable-http
    url: http://host.docker.internal:7891/mcp
    timeout: 60000

mcpSettings:
  allowedAddresses:
    - host.docker.internal:7891
```

SAFETY: Docker SSRF allowlist

LibreChat blocks private / internal addresses by default as SSRF protection. Without the `mcpSettings.allowedAddresses` entry, your MCP server will appear “connected” but every

tool call will silently fail.

On Linux, `host.docker.internal` doesn't resolve out of the box. Either add `extra_hosts: ["host.docker.internal:host-gateway"]` to the LibreChat service in your `docker-compose.yml`, or swap the URL for your host's LAN IP and allowlist that IP:port instead.

Optional: enable the LLM API path

The default behavior of every reasoning tool is `mode: chat` — the server returns the rubric + context, your chat client does the thinking, you call the tool again to persist. No LLM key needed.

If you want server-side scoring (the `batch_evaluate` tool, or `mode: api` on tools that accept it), set one of:

```
export MCP_JSA_LLM_PROVIDER=gemini
export GEMINI_API_KEY=...           # free tier covers ordinary single-user usage

# or
export MCP_JSA_LLM_PROVIDER=deepseek
export DEEPSEEK_API_KEY=...
```

`cost_estimate` reports your cumulative spend per provider per tool.

Optional: data imports for the outreach + visa features

These are *fully optional*. The system works without them.

- **LinkedIn network** — download your data export at LinkedIn → Settings → Data Privacy → Get a copy of your data → Connections. Call `import_linkedin` with the path to `Connections.csv`. Unlocks `find_warm_intros` and `find_founders`.
- **H1B visa signal** — download a DOL OFLC quarterly LCA disclosure CSV from dol.gov. Call `import_h1b`. Unlocks `visa_signal` (returns a friendliness band per company).

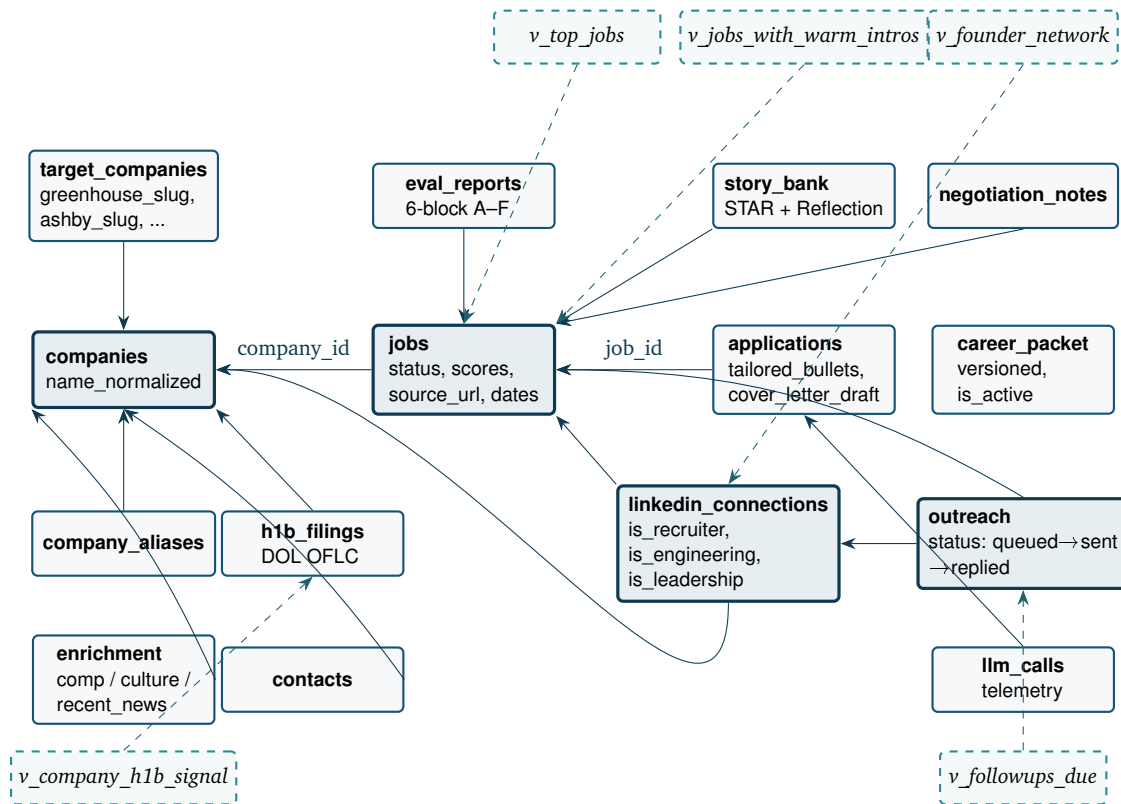
SAFETY: Personal data

Your SQLite DB now contains LinkedIn connections and H1B filings indexed against companies you've rated. The `data/` directory is gitignored by default. Don't commit it. Don't share it.

4 The data model

One SQLite file, 18 tables, 8 views. Two diagrams: the relational shape, and the job lifecycle.

4.1 Relational shape



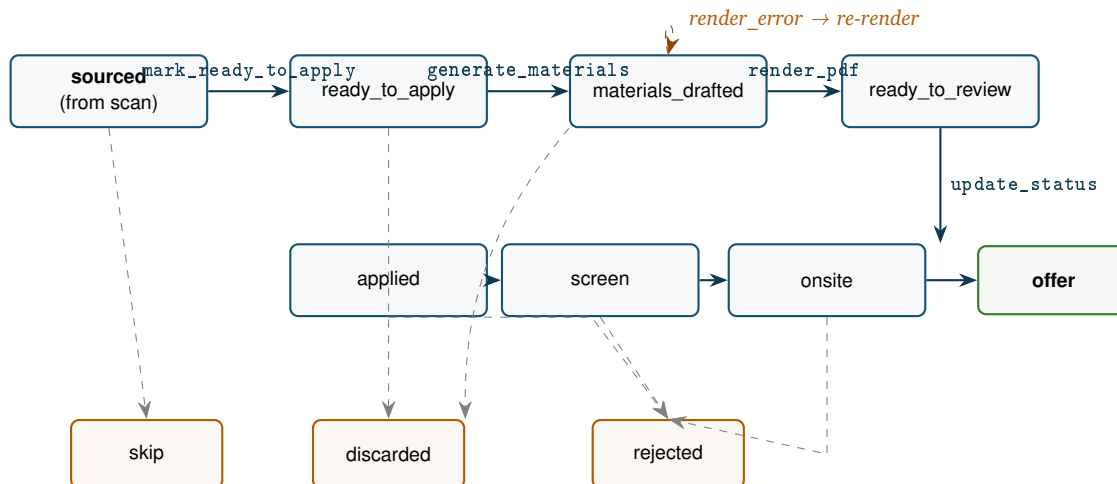
Bold-bordered tables are the ones you'll touch most often via tools. *Italic dashed boxes* are views — they're virtual; the tool layer reads from them. Lines are foreign keys; arrows point at the referenced side.

A few non-obvious shapes:

- **Cross-source dedup.** `jobs.content_hash` (sha-256 of company + title + location) is `UNIQUE`. The same role posted on Greenhouse and a company's own careers page collapses to one row.
- **Versioned career_packet.** `update_career_packet` bumps a new row with `is_active = 1` and demotes the previous active row. The history is retained — you can audit which packet generated which materials via `applications.materials_v`.
- **H1B fuzzy join.** `h1b_filings.employer_name_raw` may not match a company's canonical name. `company_aliases` captures legal-name variants; `visa_signal` resolves via the alias table when the direct join misses.
- **Strict-JSON parse log.** Every LLM call lands in `llm_calls` with `parse_ok = 0` and the raw text on failure. `cost_estimate` surfaces parse-error counts so you can tune prompts when a provider drifts.

4.2 The job lifecycle (state machine)

Every job lives in exactly one of these states. The transition rules are enforced by the `update_status` tool and a SQL CHECK constraint; the dashboard at `/` shows the current count per state.



The happy path is left-to-right across the top. **Green** is the only success terminal. **Orange** are the three discard terminals — all reachable from any state via `update_status`. The dashed arrows are weak transitions: you can drop out anywhere, but you can't normally go backwards (re-running `generate_materials` nulls the PDF cache and moves you back to `materials_drafted` for re-render).

5 Tool reference

37 tools, grouped by what they do. For each: what it does, key parameters, what it returns. The default mode for tools that take one is `chat` — the chat client is the brain; `api` mode is the LLM-direct path and needs an API key.

Evaluation

`evaluate_job`
Two-step JD evaluator.

- **Step 1:** pass input (a URL or pasted JD text). Returns `job_id`, the normalized JD, the rubric, the report format, and the active career packet for the chat to score.
- **Step 2:** pass `job_id` + report (6 blocks) + scores. Persists the eval row, updates job scores, returns the report URL.
- `mode`: `api` runs both steps server-side via the configured LLM, with strict-JSON parsing.

`batch_evaluate`

(api only) Rates all unrated jobs through the LLM. Returns A–F tier distribution + parse-error count. Strict-JSON — on parse failure, `score_total` stays NULL with the raw text logged.

`get_top_jobs`

Triage view. Filter by `min_score` (default 75), `role_category`, `status`, `limit`. Returns per-job:

scores, role, seniority, location, source URL, and the eval report link.

`evaluate_training`

Score a course / cert input on `skill_gap_fill`, `market_signal`, `time_to_value`, with a recommendation.

`evaluate_project`

Score a portfolio-project idea on `resume_signal`, `interview_story_value`, `effort_weeks`, `shipping_risk`.

Materials

`generate_materials`

Picks bullets / tagline / projects from `career_packet.md` per the JD via the tailoring rules. Writes `applications.tailored_bullets` + `cover_letter_draft`, validates against the visa rail before persisting. Two-step chat shape; api mode auto-validates LLM output.

`render_pdf`

Produces resume + cover in any subset of {pdf, tex, docx} via the `formats` param (default ["pdf"]). PDF is HTML→Chromium; `.tex` is a self-contained pdflatex source you can edit and recompile; `.docx` is generated with the docx library for Word / Google Docs editing. All three share the same tailored content. URLs persist onto the application row's `rendered_files` JSON. The visa-leakage rail runs against every output before any file is written. Files land under `/files/pdfs/`, `/files/tex/`, `/files/docx/`.

`get_report`

Returns the localhost URL + metadata for the latest `eval_report` of a `job_id`.

Sourcing

`scan_portals`

Fans out across every enabled entry in `portals.yml`. 8 providers: `greenhouse`, `ashby`, `lever`, `workday` (JSON APIs); `amazon` (JSON); `google` + `playwright_generic` (Playwright). Optional `sources` / `companies` / `title` + `location` filters. Returns counts (`found` / `new` / `dupes`) and the top 25 new rows.

Tracker

`get_tracker`

Full pipeline snapshot as JSON. Optional status filter. Mirrors the dashboard at `/`.

`update_status`

Move a job to a new status. Stamps `applied_at` on the applied transition. Optional note appended to `score_detail.status_history`.

`mark_ready_to_apply`

Bulk-marks job IDs as `ready_to_apply`. Skips terminal-state rows.

Outreach

`find_warm_intros`

Joins `jobs × non-recruiter` LinkedIn connections at the same company. Filter by `company` substring and `min_score`. Sorted by score then contact priority (engineering peers → leadership → others).

`find_founders`

Founders / CEO / CTO / c-suite from your network, derived via `v_founder_network`. Filter by kind. Stealth-company rows are flagged but excluded by default.

`draft_outreach`

Two-step chat: step 1 returns connection + job context + outreach tone rules; step 2 takes the chat-drafted message and persists it after validation (char cap, no emojis, no exclamation marks, no “refer me”, no visa mentions). `api` mode does the LLM call inline.

`draft_followup`

1–2 line nudge for sent-but-unanswered DMs. Same safety rails. Capped at 300 chars.

`draft_reply`

Contextual draft when someone has replied. Takes the received reply text, returns a tone-matched response for human editing. Cap 800 chars.

`get_outreach_queue`

Filter by status (default: `queued` / `drafted` / `edited`). Joins connection + company + linked job.

`update_outreach`

Move an outreach row to a new status. Stamps `sent_at` on sent; sets `followup_due_at` = `now` + 5d unless overridden. Re-validates `edited_message` against the safety rails.

`get_followups_due`

Returns outreach in status sent with `followup_due_at` ≤ `now`.

Visa (optional, can be hidden globally)

`visa_signal`

H1B friendliness band per company: `strong` | `mixed` | `weak` | `none`. Computed from filings count + recency. **Internal only** — never surfaced in any resume, cover letter, or outreach.

`import_linkedin`

Bulk-load `linkedin_connections` from the standard LinkedIn `Connections.csv` export. `dry_run` flag.

`import_h1b`

Bulk-load `h1b_filings` from a DOL OFLC quarterly LCA CSV. Maps employer names to companies via the alias table; unmatched names persist with `employer_id` = `NULL`.

When `MCP_JSA_VISA_SCORING=false`, all three visa tools are hidden from `tools/list` and the score formula renormalizes to $0.6 \cdot \text{resume} + 0.4 \cdot \text{taste}$. See §8.

Interview / offer

`extract_stories`

Parses STAR + Reflection rows out of the latest `eval_report.block_interview` for a job and

appends to `story_bank`. Pass an explicit `stories` array to skip parsing.

`get_story_bank`

Returns accumulated stories, with an optional competency substring filter on the tag array.

`negotiation_brief`

Returns the negotiation playbook + the most recent comp enrichment for the company + a draft framework (anchor / pillars / knobs / hard rules).

Profile + ops

`get_career_packet`

Returns the active packet (id, version, markdown content, source-cv hash).

`update_career_packet`

Replaces the active packet with a new version. Bumps version, demotes the previous `is_active`.

`reseed_career_packet`

Rebuilds the packet from the current `cv.md` + `config/profile.yml`. Same as the CLI `reseed` command. Bumps version, demotes the previous active row (history kept). Use after editing `cv.md`. Refuses to seed identity-only unless you pass `confirm_empty_cv: true`.

`enrich_company`

Persists a chat- or api-provided enrichment summary for one (company, kind) pair with a 30-day TTL.

`deep_research`

Company brief — aggregates current enrichments + top jobs + warm intros + H1B signal. With notes + kind provided in api mode, summarizes the notes into a single enrichment row.

`daily_digest`

Morning summary — new top jobs since last digest, follow-ups due, recent status changes, cost snapshot. Stamps `digest_state.last_digest_at` unless `dry_run`.

`cost_estimate`

LLM cost aggregated by provider / model / tool over the last N days (default 30). USD estimate based on a built-in rate table.

Apply (preview only)

`apply_prefill`

Opens the application URL in Chromium, detects common form fields (name, email, phone, LinkedIn, GitHub, cover letter, etc.), drafts values from your profile + tailored materials, takes a full-page screenshot. Returns the preview JSON + the screenshot URL + your resume / cover PDF URLs. **Never submits.**

Scheduler (opt-in cron, off by default)

`scheduler_status`

Lists enabled jobs, available jobs + intervals, and the last fire time.

`scheduler_enable`

Takes an array of job names from: `scan_portals_4h`, `batch_evaluate_30m`, `followups_due_1h`,

daily_digest_morning. Overrides the previous set.

scheduler_disable

Clears the enabled-jobs list. Process keeps running.

6 Worked end-to-end examples

These walkthroughs use a fictional candidate — the new graduate “Casey Riley” targeting Builder PM / Applied AI Engineer roles. Substitute your own profile mentally; the chat client wiring is identical.

6.1 Example A: paste a JD, get a tailored PDF, mark applied

Time required: 8–15 minutes.

You:

I want to apply to this role. Evaluate it, give me a tailored resume + cover, and put it in my tracker.

```
Builder PM / Technical PM - Pinch AI (Remote, US)
We're building developer tools for AI agents and need a hands-on PM who can write PRDs,
ship features end-to-end, and partner with engineering on agentic workflows, evals,
and LLM observability. 4+ years PM, strong SQL + Python, familiarity with LLM systems.
Sponsor visas. Comp $180-260k.
```

Chat:

Calling `evaluate_job` with the pasted JD...

Server:

Returns `job_id: 9a1b...`, the normalized JD, and three resources to score against (`modes/rubric.md`, `modes/report_format.md`, the active career packet).

Chat:

Reading the rubric and your packet. Scoring:

<code>resume_fit</code>	84	Strong on PRDs, SQL, Python; agentic experience via the <code>Vector Agents</code> side project.
<code>taste_fit</code>	90	Lean team, AI tooling, ships fast — matches <code>narrative.likes</code> .
<code>visa_fit</code>	88	Explicit sponsorship; comp band well above the OPT floor.
<code>score_total</code>	87	$[0.5 \cdot 84 + 0.3 \cdot 90 + 0.2 \cdot 88] = \text{A-tier.}$

Drafting the 6 blocks now...calling `evaluate_job` step 2 with `report + scores`.

Server:

Persists the eval report, renders the HTML.

```
{
  "step": "finalized",
  "job_id": "9a1b...",
```

```
"report_id": "c4d2...",
"report_url": "http://127.0.0.1:7891/files/reports/c4d2....html",
"tracker_url": "http://127.0.0.1:7891/"
}
```

Chat:

Report ready: <http://127.0.0.1:7891/files/reports/c4d2....html>. Now generating tailored materials. Calling generate_materials...

Server:

Returns the career packet + tailoring rules.

Chat:

Picking tagline option D (“Builder PM with engineering teeth”), foregrounding your e-commerce BI ownership and Vector Agents project, dropping the academic projects. Drafting the cover body (282 words). Calling generate_materials again to persist... then render_pdf kind=both...

Server:

```
{
  "files": [
    {"kind": "resume", "url": "http://127.0.0.1:7891/files/pdfs/resume-builder-pm-9a1b.pdf", "bytes": 271769},
    {"kind": "cover", "url": "http://127.0.0.1:7891/files/pdfs/cover-builder-pm-9a1b.pdf", "bytes": 61464}
  ]
}
```

Chat:

Both PDFs at the links above. Click resume and cover to inspect. When you’ve submitted via Pinch’s portal, say so and I’ll update the tracker.

You:

Done, submitted. Mark it applied.

Chat:

Calling update_status job_id=9a1b... status=applied...done. Pipeline now shows: **1 applied, 1 onsite, 2 ready_to_apply**. Open the tracker at <http://127.0.0.1:7891/>.

EXAMPLE: what just happened

Two MCP tool calls (evaluate_job twice, render_pdf once, update_status once), one transcript, four artifacts: the 6-block report, the resume PDF, the cover PDF, an updated tracker row. The chat did the writing; the server did the persistence + rendering. You did the

deciding and submitting.

6.2 Example B: scan portals, triage, batch evaluate

Time required: 5 minutes of human attention, 3 minutes of background LLM work.

You:

Scan my portals and rate everything new. Then show me the top PM jobs.

Chat:

Calling scan_portals...

Server:

```
{
  "companies_scanned": 18,
  "jobs_found": 412,
  "jobs_new": 47,
  "jobs_dupes": 5,
  "sources": ["greenhouse", "ashby", "lever"],
  "errors": []
}
```

Chat:

47 new jobs after title/location filtering. Now batch_evaluate on the unrated set...

Server:

```
{
  "rated": 45,
  "parse_errors": 2,
  "distribution": {"A":4, "B":11, "C":17, "D":10, "F":3, "unrated":2}
}
```

Chat:

A/B-tier yield: 15 out of 47 (32%). Two parse errors will retry on the next batch run. Pulling top PM jobs... calling get_top_jobs role_category=pm min_score=80...

Server:

```
{
  "count": 6,
  "items": [
    {"title": "Builder PM", "company_name": "Vellum", "score_total": 91, "location": "Remote", "US", "report_url": "http://.../reports/...html"},
  ]
}
```

```
{
  "title": "AI Product Manager", "company_name": "Mosaic", "score_total": 88, "location": "NYC", "report_url": "http://.../..."},
  {"title": "Technical PM, Agents", "company_name": "Loop AI", "score_total": 86, "location": "Remote", "report_url": "http://.../..."}
}
```

Chat:

6 PM jobs $\geq$ 80. Click any report link to read the 6-block evaluation. Want me to mark the top 3 ready_to_apply and start generating materials?

EXAMPLE: the triage instinct

The brief 5-minute version is enough. You don't need to read all 45 reports. Skim the top 3–5, kill the false positives with `update_status status=skip`, and queue the keepers for tomorrow's material-generation pass. The whole point of the rubric is to push the human-attention budget toward the top of the funnel.

6.3 Example C: find a warm intro, draft outreach, follow up

Time required: 4–7 minutes per DM. Prereq: `import_linkedin` run once.

You:

I scored 91 on the Vellum Builder PM role. Do I know anyone there?

Chat:

Calling `find_warm_intros company=Vellum...`

Server:

```
{
  "count": 2,
  "items": [
    {"full_name": "Priya Subramaniam", "position": "Staff Engineer, Agents", "contact_priority": 1, "connection_id": "abc-1", "linkedin_url": "https://linkedin.com/in/priya-test"},
    {"full_name": "Jordan Lee", "position": "Head of Product", "contact_priority": 2, "connection_id": "abc-2", "linkedin_url": "https://linkedin.com/in/jordan-test"}
  ]
}
```

Chat:

Two: Priya (Staff Eng) and Jordan (Head of Product). Priya's the engineering-peer angle — the tone rules say lead with a technical question. Calling `draft_outreach connection_id=abc-1 job_id=<vellum> type=warm_intro_request...`

Server:

Returns connection + job context + the active career packet + modes/outreach_tone.md.

Chat:

Drafting at 287 chars (cap is 600). Reviewing against the rails: no emojis, no exclamation marks, no “refer me”, no visa mention, soft close. Calling draft_outreach again with the message to persist...

Server:

```
{
  "outreach_id": "o-12",
  "status": "drafted",
  "validation": {"ok": true, "message_len": 287, "limit": 600, "type": "warm", "issues": []}
}
```

Chat:

Drafted. Here's the message — edit before sending:

saw your recent post about agent eval harnesses — curious how y'all are thinking about multi-step trace replay. i've been shipping agentic workflows on AutoGen + LlamaIndex at my last role, and the trace problem is the one we kept hitting. would love a 15-min chat if it lines up. no worries if not a fit.

When you send, tell me and I'll update the row + queue a follow-up for 5 days out.

You:

Sent.

Chat:

update_outreach id=o-12 status=sent — done. Follow-up due 5 days from today.
(5 days later, no reply.)

get_followups_due shows o-12. Calling draft_followup id=o-12...

Server:

Returns the original message + days-since-sent for context.

Chat:

Drafting 1-line nudge (98 chars):

still curious about your take on trace replay — happy to come back if it ever makes sense.

EXAMPLE: tone rules at work

The draft passed all rails: under 600 chars, no exclamation marks, no emoji, no “refer me”, no visa or sponsorship reference, leads with a specific technical observation, drops one credibility marker (“shipping agentic workflows on AutoGen + LlamaIndex”), and closes soft. The follow-up nudge keeps the same tone in one line. Both will sound like a peer-to-peer DM, not a job application.

7 The daily operating ritual

TIP

This is the most important section of the guide. The system pays back in repeated use, not one-shot setup. A polished evaluation that never becomes a sent application is worth nothing.

A realistic split for someone running a serious search — **10–30 minutes a day**, three time-boxed slots.

Morning: triage (10 minutes)

1. Open the chat. *“Run daily_digest.”*
2. Read the digest: new top jobs since yesterday, follow-ups due today, status changes in the pipeline.
3. For each new A-tier job: open the report link, skim the 6 blocks. **One of three actions:** `mark_ready_to_apply`, `update_status status=skip`, or do nothing (decide tomorrow). *Do not let A-tier jobs sit in sourced for more than 48 hours — they go stale.*

Midday: send (15 minutes)

1. *“get_followups_due.”* For each row: `draft_followup`, `edit`, `send`, `update_outreach status=sent`.
2. *“find_warm_intros for the top 3 ready_to_apply jobs.”* For each non-trivial match: `draft_outreach`, `edit`, `send`, `update_outreach status=sent`.
3. Anyone replied? *“Show outreach in replied status.”* For each: `draft_reply`, `edit`, `send`.

Evening: apply (15 minutes)

1. Pick **one** job from `ready_to_apply`. `generate_materials`. Review the tailored bullets in chat.
2. `render_pdf kind=both`. Open both URLs, eyeball the PDF.
3. *“apply_prefill for this job.”* Open the screenshot URL, see the draft form values. Open the actual application URL in a new tab, paste + upload, submit manually.
4. `update_status status=applied`. Done for the day.

Weekly: review (20 minutes, e.g. Sunday)

1. *“Show the tracker.”* Eyeball the funnel — where are you stuck?

2. “*Show the story bank.*” Did this week’s interviews add new stories? Tag them by competency.
3. “*cost_estimate.*” Pennies for the LLM. Sanity-check before the next batch.
4. Edit `modes/rubric.md` or `config/profile.yml` if last week’s matches were systematically off. The system gets sharper with iteration.

The bottleneck is human attention, not the tool. 30 well-targeted applications with warm intros over 4 weeks outperforms 200 cold blasts. Resist the temptation to optimize the system instead of using it. If you find yourself fiddling with the rubric weights instead of clicking *send*, that’s the failure mode — close the IDE, open the chat.

8 Customization

Every behaviour-shaping piece is a Markdown file or a config entry. No code changes required.

Editing the modes

The six files in `modes/` are exposed as MCP resources and live-reloaded on edit (the server watches the directory). After saving, the chat picks up the change on the next `resources/read` — no restart.

File	What it controls
<code>modes/rubric.md</code>	Scoring dimensions, weights, role-priority order, archetype-override rule, hard rules
<code>modes/report_format.md</code>	6-block A–F (+G) report shape
<code>modes/tailoring_rules.md</code>	Resume/cover tailoring decisions per <code>role_category</code>
<code>modes/outreach_tone.md</code>	Warm-intro / founder DM char caps, forbidden phrases, persona rules
<code>modes/negotiation_playbook.md</code>	Negotiation framework + scripts (anchor, pillars, knobs)
<code>modes/career_packet.md</code>	Your bullet/project bank — the superset of every claim

Disabling visa scoring (fully optional)

Visa signal exists because the system’s original author was on OPT/STEM-OPT and needed to filter for visa-friendly employers. If you don’t need that — US citizens, non-US users, anyone scoring roles where sponsorship is irrelevant — set:

```
export MCP_JSA_VISA_SCORING=false
```

When disabled, the server:

- Drops `visa_fit` from the rubric. `score_total = round(0.6 · resume_fit + 0.4 · taste_fit)` — enforced server-side regardless of what the chat returns.
- Prepends a “VISA SCORING DISABLED” override block to the top of the `mcp-jsa://modes/rubric` resource.
- Hides `visa_signal`, `import_h1b`, `import_linkedin` from `tools/list`.

- Strips `score_visa_fit` from `get_top_jobs` items and the eval-report HTML badge.
- Records `visa_scoring_enabled: false` in every `score_detail` blob so you can audit which formula scored a given row.

Every other feature is unaffected.

Adding target companies

Append to `portals.yml` `tracked_companies:` — restart the server (or wait for the next `scan_portals` call which re-reads the file).

```
- name: "Vellum"
  careers_url: "https://job-boards.greenhouse.io/vellum"
  priority: 1
  enabled: true

- name: "ExampleCo"
  workday_url: "https://exemplco.wd5.myworkdayjobs.com/External"
  priority: 2
  enabled: true

- name: "ClosedBoardCo"
  provider: "playwright_generic"
  careers_url: "https://example.com/careers"
  selectors:
    item: "a.job-card"
    title: ".job-title"
    location: ".job-location"
    wait_for: ".jobs-loaded"
  enabled: false
```

Non-US markets

The defaults assume nothing. To localize:

- Translate the headings of `modes/rubric.md` and `modes/negotiation_playbook.md`; keep the formula structure.
- Update `narrative.likes / dislikes` and `compensation.target_range` in `config/profile.yml` to match your market.
- Add region-specific portals to `portals.yml`. The generic Playwright provider works against any HTML careers page given the right selectors.
- Localize `title_filter.positive / negative` — “Ingeniero” for Spanish-language postings, “Ingénieur” for French, etc.

Tuning the rubric weights

Edit `modes/rubric.md`. Change the weights paragraph and the formula. Next `evaluate_job` call returns the new rubric; the chat will use the updated math.

TIP

If you change weights mid-search, your existing `score_total` values reflect the old formula. `batch_evaluate` will re-rate any job whose `score_total` is `NULL` — set the column to `NULL` for everything you want re-scored.

9 The hard rules (safety model)

Five rules are enforced at the code level, not just documented. Removing any of them is a one-line change you'd notice in code review.

SAFETY: Rule 1: Visa data never reaches a candidate-facing artifact

`scanForVisaLeakage()` runs inside `render_pdf` (on `cover_body`) and inside `generate_materials` (on the entire serialized output) before persistence. If a leak is detected the call **fails**, not warns. Visa data is internal to `visa_signal` scoring and `visa_fit` — that's the only valid surface.

SAFETY: Rule 2: Never invent claims outside the career packet

The materials generator instruction set explicitly tells the chat: “NEVER invent metrics not in the packet.” The packet is the authoritative source; bullets that cite numbers not present in it should fail human review. `update_career_packet` bumps a version so the audit trail is permanent.

SAFETY: Rule 3: Human-in-the-loop everywhere

No tool auto-submits an application. `apply_prefill` opens the page, drafts values, takes a screenshot, and stops — the human submits manually. No tool auto-sends a DM. `draft_outreach` persists a draft; the human sends, then calls `update_outreach status=sent`. Removing this gate is the single change that would turn this from a tool into a spam cannon.

SAFETY: Rule 4: Strict-JSON parsing with visible failure

Every api-path LLM call uses `response_format: json_object`. On parse failure the call records `parse_ok = 0` + the raw text in `llm_calls`, and the score row stays `NULL` — *never* a default (0,0,0,0). “Silent zeros” was the JSA pipeline’s worst failure mode; `cost_estimate` surfaces parse-error counts so drift is visible.

SAFETY: Rule 5: Serialized writes

Every tool that mutates the tracker / applications / outreach goes through `runInWriteLock()` in `src/db.ts`. Concurrent tool calls won't interleave their writes; the state machine stays consistent.

The outreach safety rails extend Rule 1 with four more pattern groups, validated in `validateOutreach()` before any DM is persisted:

Rule	What it rejects
char_cap	Warm: 600 chars. Founder: 300. Follow-up: 300. Reply: 800.
no_visa_mentions	H1B / OPT / EAD / visa / sponsorship / green card / work permit / citizenship.
no_refer_me	“refer me” / “put my resume in front of” / “good word” / “get me an interview”.
no_emojis	Any emoji codepoint.
no_exclamation_marks	Any !.
no_cliches	“I hope this finds you well” / “I’d love to pick your brain”.

A failing draft is returned to the chat with the specific rule + offending hit so it can be revised. The drafter never silently strips and saves.

10 Troubleshooting and FAQ

Setup issues

“Chromium isn’t installed” on first start

`npx job_ops-mcp start` auto-installs Chromium on first run (150 MB, 30 s). If you skipped it via `-skip-chromium-check` or the install errored, run `npx playwright install chromium` manually and re-try.

“Tool not found” from the chat client

Your client connected before the server had registered tools. Check the connection mode — for Claude Desktop, the command: `npx block` should spawn the server inline with the `-stdio` flag; for LibreChat, the server must already be running on the URL you specified. Run `npx job_ops-mcp doctor` to confirm.

LibreChat shows “connected” but every call fails

You’re in Docker, the LibreChat container can’t reach your host’s 127.0.0.1. Swap to `host.docker.internal` **and** add the entry to `mcpSettings.allowedAddresses`. On Linux, add `extra_hosts: ["host.docker.internal:host-gateway"]` to `docker-compose.yml`.

LLM tools error with “No LLM provider configured”

Set `MCP_JSA_LLM_PROVIDER` + the matching API key. The default chat mode of every tool works without one — only `batch_evaluate` and `mode: api` need a key.

Migrations didn’t run

The server applies pending migrations on first DB open. If the `data/` directory is on a read-only filesystem, the boot will error. Set `MCP_JSA_DATA_DIR` to a writable path and re-run.

Doctor says “cv.md missing” but it’s in the directory

You probably launched from a different CWD than where `init` ran. Set `MCP_JSA_PROJECT_ROOT` to the absolute path, or `cd` into the right directory before `start`.

Conceptual FAQ

Does this work without visa data?

Yes. `MCP_JSA_VISA_SCORING=false` drops the visa surface entirely — formula renormalizes, tools hidden, columns stripped. Use the system exactly as documented; only the visa-specific bits go quiet.

Does this work outside the US?

Yes. The defaults assume no specific market. Edit `modes/*.md` to match your local language / comp norms, add region-specific portals to `portals.yml`, and adjust `title_filter`.

How much does an LLM key cost?

On Gemini 2.5 Flash (free tier), a single `batch_evaluate` on 50 fresh jobs is comfortably under the per-minute quota and the per-day quota. `cost_estimate -days 30` reports your actual usage. The chat-mode tools cost zero on the server side (your chat client pays its own inference cost via your subscription).

Why isn't there an `auto_apply` tool?

That's the line. Every irreversible boundary — submit, send DM, post comment — is a human action. The system makes the prep cheaper; the human still decides and acts. See §11.

Can I run two of these on the same machine?

Yes — pick different ports and project roots. Each gets its own SQLite file; the two never interact.

Where does the SQLite file go?

By default, `<project_root>/data/mcp-jsa.db`. Override with `MCP_JSA_DATA_DIR`.

How big does the DB get?

After 6 months of daily use with 10–20k jobs scored, expect 50–100 MB. SQLite handles that comfortably with WAL mode. Back up by copying the directory; nothing else.

Will the schema change?

Yes, additively. Future migrations land in `src/migrations/00X_...sql` and apply on first DB open after `npm update`. The migration ledger lives in the `_migrations` table so already-applied ones aren't re-run.

11 This finds jobs. It doesn't send them.

A frank coda.

`job_ops-mcp` is good at four things:

1. **Sourcing.** Polling 20+ ATS endpoints in a single `scan_portals` call, with cross-source dedup. Better than refreshing tabs by hand.
2. **Scoring.** A consistent rubric applied to every JD, persisted with the reasoning. Better than re-deciding “is this worth my time?” a hundred times.
3. **Drafting.** Tailored bullets, ATS-clean PDFs, on-tone DMs, follow-up nudges. Better than starting from scratch each time.
4. **Tracking.** One database, one tracker URL. Better than a spreadsheet that drifts.

It is **not** good at — and is intentionally architected against — the thing that actually closes the loop: *deciding which roles to apply to and sending the application or the DM*. That is the human's job, every day. The system can score 50 jobs in an evening; you can probably honestly apply to two or three a day if the materials are tailored and the outreach is real. The bottleneck is the human, by design.

If you find yourself **tweaking the rubric instead of clicking send**, the system is failing you — not because it's broken, but because it's optimizing the wrong axis. The daily ritual in §7 is built around that risk. Use it.

Good luck out there.